Restaurant Management System – Developer Documentation

1. Project Overview

The Restaurant Management System is a desktop application built using **Qt (C++)** with a **MySQL** database backend. The software is designed to manage restaurant operations such as order handling, staff roles, menu management, and automatic database backup.

The system supports multiple user roles including Admin, Order Manager, Owner, and Kitchen Staff. Each role has its own interface and responsibilities within the application.

---

2. Technology Stack

Frontend Framework: Qt (C++) Database: MySQL Database Tool: MySQL Server IDE: Qt Creator Operating System: Windows

Main Libraries Used:

- QtWidgets
- QtSql
- QProcess
- QFile
- QDir
- QTimer

---

3. System Architecture

The system follows a modular window-based architecture:

Login Window | |---- Admin Panel |---- Order Panel |---- Owner Panel |---- Kitchen Panel

Each panel interacts with the MySQL database for performing operations.

---

4. User Roles

4.1 Admin

Responsibilities:

- Add / Edit / Delete Menu Items
- Manage Staff
- View Reports
- Restore Database Backup

4.2 Order Staff

Responsibilities:

- Create Orders
- Manage Dine-In / Takeaway Orders
- Send Orders to Kitchen

4.3 Owner

Responsibilities:

- View Sales Reports
- Monitor Orders

4.4 Kitchen

Responsibilities:

- View Incoming Orders
- Update Order Status

---

5. Database Structure

Database Name: restaurant_management

Main Tables:

menu

Stores all menu items.

Columns:

- id
- item_name
- price
- image_path

staff

Stores employee information.

Columns:

- id
- name
- role
- username
- password

orders

Stores order information.

Columns:

- order_id
- order_type
- total_price
- order_time

order_items

Stores items belonging to each order.

Columns:

- id
- order_id
- menu_id
- quantity

---

6. Image Handling

Menu item images are stored as **file paths in the database**.

Example:

F:/RestaurantSystem/Icons/chicken_biryani.jpg

When loading menu items, the application reads the image path from the database and loads the image using Qt.

---

7. Automatic Database Backup

The system automatically creates database backups using mysqldump.

Backup Location:

Restaurant-Managment-System/backup/

Backup Naming Format:

backup_YYYY_MM_DD_HH_MM.sql

Example:

backup_2026_03_10_15_48.sql

A QTimer runs every hour (or configured interval) to generate a new backup.

Old backups are automatically deleted keeping only the latest 5 backup files.

---

8. Backup Implementation

The system uses QProcess to execute mysqldump:

mysqldump --set-gtid-purged=OFF -u root -pPASSWORD restaurant_management

The SQL output is captured and saved into a .sql file.

---

9. Restore System

The Admin panel provides a Restore button.

Process:

1. Admin selects a .sql backup file
2. Application reads the SQL file
3. SQL commands are sent to mysql.exe using QProcess
4. Database is restored

---

10. Backup Safety Mechanism

To prevent storage overflow:

Only the latest **5 backup files** are stored.

Older backups are automatically deleted using QDir.

---

11. Security Notes

Passwords are passed through command-line arguments when executing mysql.exe.

MySQL may show the warning:

"Using a password on the command line interface can be insecure"

This is normal for local applications and does not affect functionality.

---

12. How to Run the Project

1. Install MySQL Server
2. Create database: restaurant_management
3. Import database schema
4. Configure database connection in Qt project
5. Build the project in Qt Creator
6. Run the application

---

13. Required External Software

Required tools:

- MySQL Server
- Qt Creator
- MinGW Compiler

MySQL executables used:

mysqldump.exe mysql.exe

Typical Path:

C:/Program Files/MySQL/MySQL Server/bin/

---

14. Folder Structure

Restaurant-Managment-System

/src main.cpp mainwindow.cpp user1window.cpp user2window.cpp user3window.cpp user4window.cpp

/icons menu images

/backup automatic SQL backups

/build compiled executable

---

15. Future Improvements

Possible enhancements:

- Cloud backup integration
- Online order system
- Mobile application
- Real-time kitchen notifications
- Sales analytics dashboard

---

16. Troubleshooting

Backup file empty

Check mysqldump path and database credentials.

Restore not working

Ensure mysql.exe path is correct.

Images not showing

Verify image path stored in database.

---

17. Developer Notes

This project is intended for educational use and demonstration of:

- Qt GUI development
- Database integration
- Multi-user software architecture
- Automated backup systems

Developers modifying the project should maintain separation between UI logic and database operations for easier maintenance.

---

18. Installation Guide for Developers

Follow these steps to set up the project on a new system:

1. Install **Qt Creator** with the MinGW compiler.
2. Install **MySQL Server**.
3. Create a database named:

restaurant_management

4. Import the initial database schema using MySQL Workbench or mysql command line.
5. Clone or copy the project folder.
6. Open the project .pro file in Qt Creator.
7. Verify database connection credentials inside the source code.
8. Build the project using the Debug or Release configuration.
9. Run the application.

If the project fails to connect to the database, ensure MySQL is running and credentials are correct.

---

19. UML Class Overview

Main classes used in the system:

MainWindow

- Handles login system
- Controls automatic database backup
- Opens role-based windows

User1Window (Admin)

- Menu management
- Staff management
- Database restore

User2Window (Order Panel)

- Create new orders
- Manage dine-in and takeaway orders

User3Window (Owner Panel)

- Displays reports
- Monitors restaurant activity

User4Window (Kitchen Panel)

- Displays incoming orders
- Updates food preparation status

Each window interacts with the database using Qt SQL classes such as QSqlDatabase and QSqlQuery.

---

20. Database ER Overview

Core relationships in the database:

menu

- id (Primary Key)
- item_name
- price
- image_path

orders

- order_id (Primary Key)
- order_type
- total_price
- order_time

order_items

- id (Primary Key)
- order_id (Foreign Key -> orders.order_id)
- menu_id (Foreign Key -> menu.id)
- quantity

staff

- id (Primary Key)
- name
- role
- username
- password

Relationship Summary:

One Order -> Many Order Items One Menu Item -> Many Order Items

This structure allows each order to contain multiple menu items while keeping menu data normalized.

---

21. Recommended Coding Practices

For developers extending this system:

- Keep UI logic separate from database queries.
- Use Qt signals and slots for communication between UI components.
- Avoid hardcoding file paths where possible.

- Store configuration values (database credentials, backup paths) in a config file in future versions.

---

22. Future Scalability Suggestions

If the system is expanded into a commercial application, consider:

- Role-based authentication using encrypted passwords
- Network-based multi-terminal order system
- Cloud backup integration
- Inventory management